

CSE 256 | Lecture 9: Self-Attention & Transformers

Ndapa Nakashole

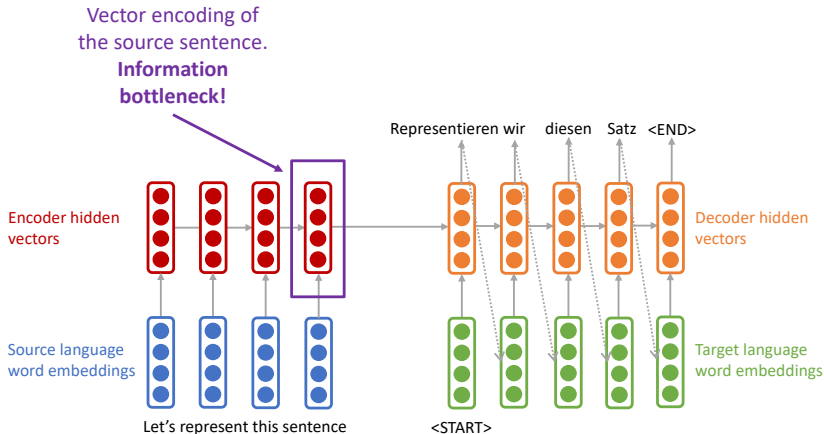
February 3, 2026

Administrative matters

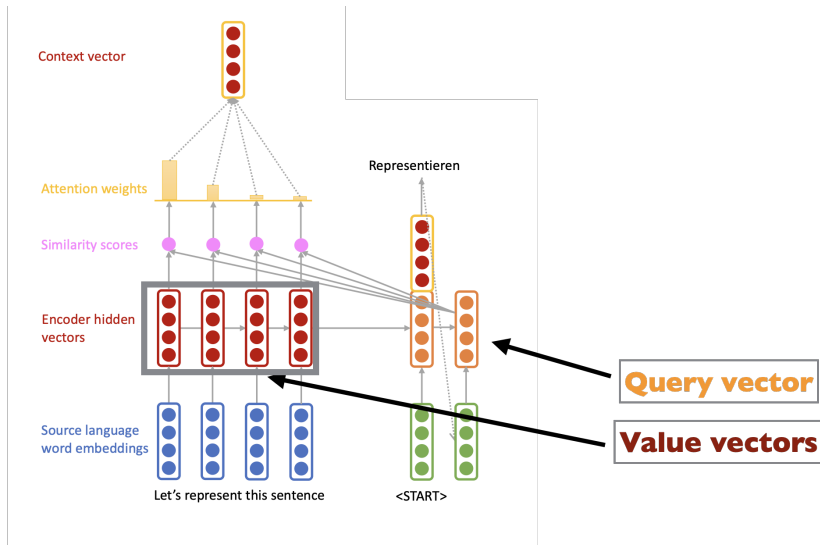
- ▶ **PA1:** last day to submit was Feb 1st
- ▶ **PA2:** now out, due Feb 20th
- ▶ **Midterm Review Exam:** available on Canvas, under Syllabus, Week 8, Lecture 16 cell

Recap: Seq2Seq

- ▶ We introduced Seq2Seq models for machine translation
 - ◉ Encoder RNN encodes source sentence into fixed-length vector
 - ◉ Decoder RNN decodes target sentence from fixed-length vector



Recap: Seq2Seq with Attention



Today

How can we represent long contexts

- ▶ without exploding **parameter counts** (concatenation),
- ▶ and without forgetting **far-back information** (RNNs)?

- 1 Self-Attention
- 2 The Transformer Architecture

Why Context Matters

Astronomers study celestial bodies

The meaning of a word depends on the other words around it.

Given a word (e.g., *astronomers*), self-attention computes a **context-sensitive** representation as a **weighted combination of all words**:

$$\mathbf{h}_{\text{astronomers}} = a_1 \mathbf{x}_{\text{astronomers}} + a_2 \mathbf{x}_{\text{study}} + a_3 \mathbf{x}_{\text{celestial}} + a_4 \mathbf{x}_{\text{bodies}}$$

- ▶ Weights reflect relevance to *astronomers*
- ▶ $\sum_i a_i = 1$ (softmax normalization)

Self-Attention: Overview

Self-attention transforms **context-free** representations into **context-sensitive** ones.

- ▶ Input: each item $\mathbf{x}_i \in \mathbb{R}^d$ is context-free
- ▶ Output: each item $\mathbf{h}_i \in \mathbb{R}^d$ is context-sensitive

Self-Attention Formalism: Queries, Keys, Values

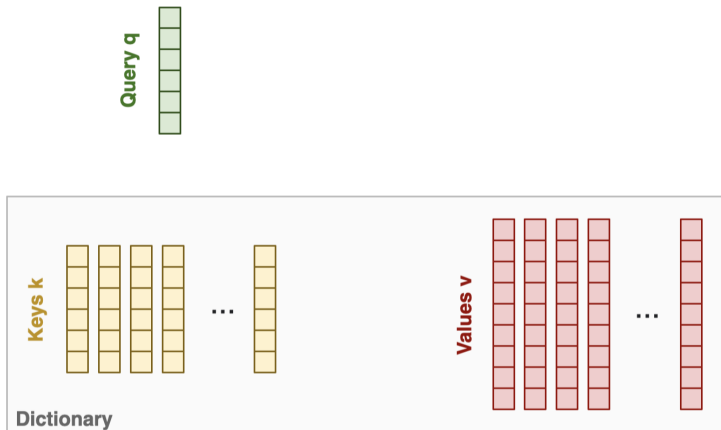
- ▶ **Given:** a **query** vector q and a set of **key–value** pairs (k_i, v_i)
- ▶ **Compute:** weights from the similarity between q and k_i
- ▶ **Output:** a weighted sum of the **values**

$$h = \sum_i a_i v_i$$

- ▶ **Interpretation:** a selective summary of the **values**,
conditioned on the **query**

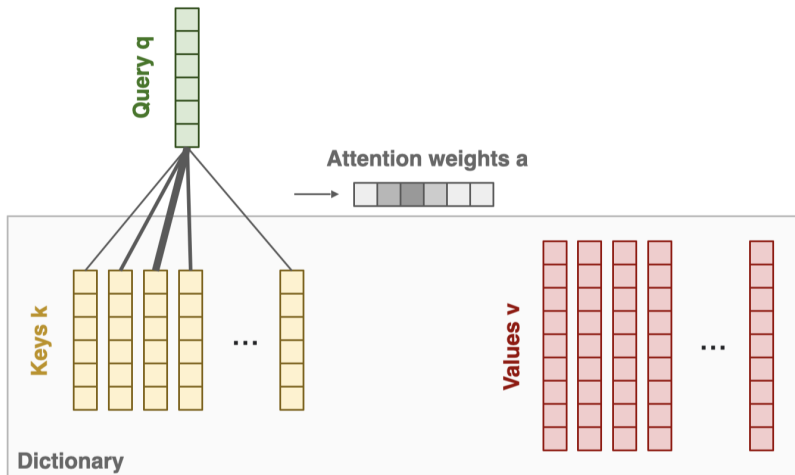
Keys provide a separate space for computing attention scores,
increasing model capacity by separating *scoring* from *content*

Self-Attention: Setup



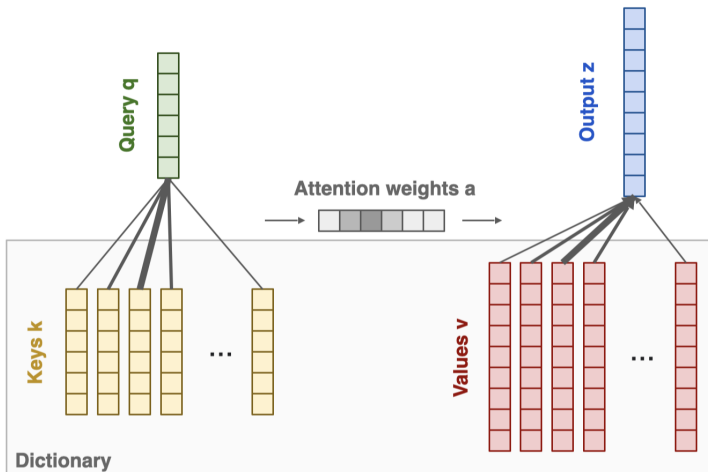
Attention begins with a query and a set of key–value pairs. We will retrieve information from the values based on the query–key similarities.

Computing Attention Scores



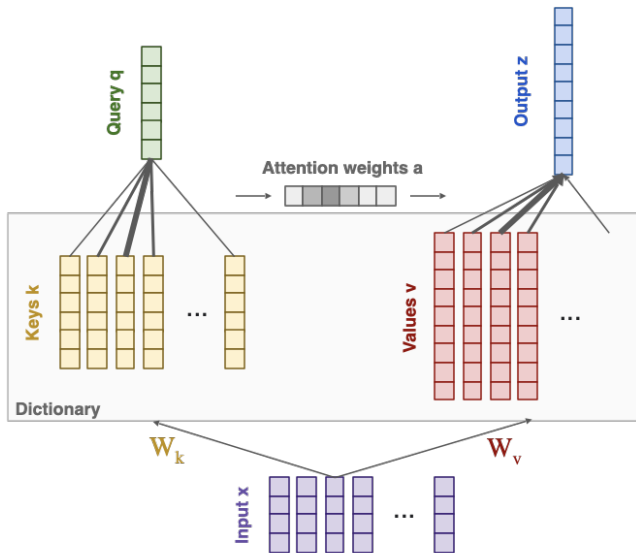
Each key is scored independently against the query, producing attention weights.

Attention Output



Attention weights select and combine the values to produce the output.

Where Do Queries, Keys, and Values Come From?



Queries, keys, and values are learned linear projections of input vectors

Where Do Queries, Keys, and Values Come From?

- Keys and values are derived from the same input x :

$$k = W_k x, \quad v = W_v x$$

- The query can come from a different input y :

$$q = W_q y$$

- If the query comes from the same input x , we call it **self-attention**:

$$q = W_q x$$

Self-Attention vs. Cross-Attention

► In seq2seq with attention:

- ⊙ Values come from the **source sentence** x
- ⊙ Queries come from the **target sentence** y

$$v = W_v x, \quad q = W_q y$$

► In self-attention (e.g., language modeling):

- ⊙ Queries, keys, and values all come from the **same sequence**

$$k = W_k x, \quad v = W_v x, \quad q = W_q x$$

Attention Mechanism (1/3): Scoring

$$\text{score}_i = \mathbf{q} \cdot \mathbf{k}_i$$

- ▶ Measures relevance of key i to the query
- ▶ Larger score $\Rightarrow$ more attention

Attention Mechanism (2/3): Normalization via Softmax

$$\alpha_i = \frac{e^{\mathbf{q} \cdot \mathbf{k}_i}}{\sum_j e^{\mathbf{q} \cdot \mathbf{k}_j}}$$

- ▶ $\alpha_i \geq 0$
- ▶ $\sum_i \alpha_i = 1$

Attention Mechanism (3/3): Aggregation via Weighted Sum

$$A(\mathbf{q}, \mathbf{K}, \mathbf{V}) = \sum_i \alpha_i \mathbf{v}_i$$

- ▶ Weighted sum of values
- ▶ Output lives in value space $\mathbb{R}^{d_v}$

Attention in Three Steps

For a single query q :

- 1 **Score:** compare the query to each **key**

$$\text{score}_i = q \cdot k_i$$

- 2 **Normalize:** convert scores into attention weights

$$\alpha_i = \text{softmax}(\text{score}_i)$$

- 3 **Aggregate:** compute a weighted sum of the **values**

$$h = \sum_i \alpha_i v_i$$

Attention Enables Parallel Computation

► RNN

$$\mathbf{h}_i = f(\mathbf{h}_{i-1}, \mathbf{x}_i)$$

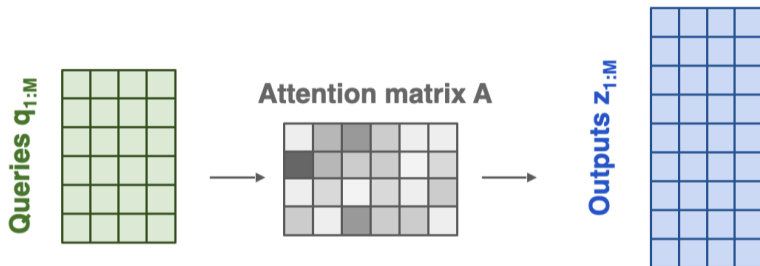
- ⊙ Each hidden state depends on the previous one
- ⊙ Computation is inherently sequential

► Self-Attention

$$\mathbf{h}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$

- ⊙ Each output depends directly on all inputs
- ⊙ All outputs can be computed in parallel
 - There is no dependence on $\mathbf{h}_{i-1}, \mathbf{h}_{i-2}, \dots$, so nothing forces a sequential order.

Matrix Notation: Multiple Queries at Once



We now have multiple queries in matrix Q , keys in matrix K , and an attention matrix A with one row per query.

Attention (Matrix Form) (1/3): Score

Compute relevance scores for all queries:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{|Q| \times |K|}$$

Queries and keys have the same dimensionality d_k :

► $\mathbf{Q} \in \mathbb{R}^{|Q| \times d_k}$

► $\mathbf{K} \in \mathbb{R}^{|K| \times d_k}$

Attention (Matrix Form) (2/3): Normalize

Convert scores into attention weights:

$$\mathbf{A} = \text{softmax}\left(\mathbf{Q}\mathbf{K}^\top\right) \in \mathbb{R}^{|Q| \times |K|}$$

- ▶ Softmax is applied **row-wise**
- ▶ Each row sums to 1

Attention (Matrix Form) (3/3): Aggregation via Weighted Sum

Compute output representations:

$$\mathbf{Z} = \mathbf{A} \mathbf{V} \in \mathbb{R}^{|Q| \times d_v}$$

- ▶ $\mathbf{V} \in \mathbb{R}^{|K| \times d_v}$
- ▶ Each output is a weighted sum of values

Where Can Things Go Wrong?

- ▶ Attention weights are produced by a **softmax**
 - ⊙ But, Softmax behavior depends on the *scale* of its inputs!
- ▶ In the single-query case:

$$q \cdot k_i$$

What happens if these dot products become very large?

Why Scaling Matters for Softmax (1/2)

The derivative of softmax:

$$\frac{\partial \text{softmax}(z_i)}{\partial z_i} = \text{softmax}(z_i)(1 - \text{softmax}(z_i))$$

- ▶ The gradient is largest when the output is uncertain
- ▶ When softmax saturates (≈ 0 or ≈ 1), gradients become very small

Large dot products push softmax into saturation.

Why Scaling Matters for Softmax (2/2)

Consider attention scores for one query:

$$\mathbf{z} = [100, 3, 2, 1]$$

After softmax:

$$\text{softmax}(\mathbf{z}) \approx [1.0, 0.0, 0.0, 0.0]$$

For the dominant entry:

$$\frac{\partial \text{softmax}(z_1)}{\partial z_1} = p_1(1 - p_1) \approx 0$$

For the others:

$$\frac{\partial \text{softmax}(z_i)}{\partial z_i} \approx 0 \quad (i \neq 1)$$

- ▶ Attention weights become nearly fixed
- ▶ Gradients w.r.t. attention scores are near zero
- ▶ Keys and queries receive almost no update signal

Scaled Dot-Product Attention

- ▶ **Problem:** Dot products grow in magnitude as the key dimension d_k increases
 - ⊙ Attention weights become nearly one-hot
 - ⊙ Gradients w.r.t. queries and keys become very small
- ▶ **Solution:** Scale the dot product by $\sqrt{d_k}$

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$$

Self-Attention: Summary

What Attention Does

- ▶ Score:

$$\mathbf{q} \cdot \mathbf{k}_i$$

- ▶ Normalize:
softmax

- ▶ Aggregate:

$$\sum_i \alpha_i \mathbf{v}_i$$

Key Properties

- ▶ Parallel computation over all queries
- ▶ Order-invariant

Learning Considerations

- ▶ Softmax must not saturate
- ▶ Scaled dot-product attention:

$$\text{softmax} \left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d_k}} \right)$$

Multi-Head Attention

What Is Attention Actually Doing?

- ▶ For each word (**query**), attention asks:
Which other words should I look at to understand my role in this sentence?
- ▶ But “relevance” is not a single notion
 - ⊙ A word can be relevant to other words for many reasons
 - ⊙ Multiple relationships may matter at the same time

If I am...	I need to know...
a verb	who is my subject ?
a verb	what is my object ?
negated	who is the negator ?
polysemous	which words clarify my sense ?
a pronoun	who is my antecedent ?

Single-Head Attention: A Representation Bottleneck

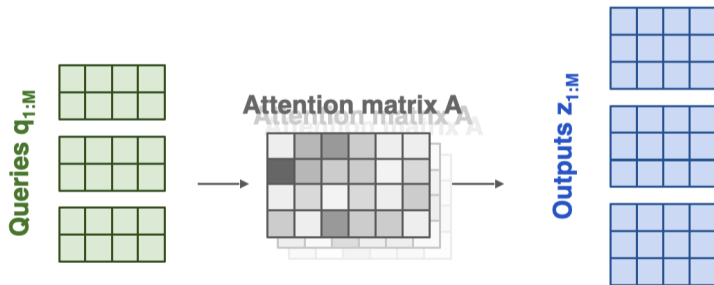
- ▶ A single attention head produces:
 - ⊙ one attention distribution, one weight per word
- ▶ Consider a polysemous, negated verb, should attention focus on the:
 - ⊙ subject?
 - ⊙ object?
 - ⊙ negator?
 - ⊙ words that disambiguate its meaning?
- ▶ **Bottleneck:** A single attention head must collapse all these different notions of relevance into one distribution.
- ▶ **Idea:** Use **multiple attention heads** so different relationships can be modeled in parallel.

Multiple Heads: What Changes, and What Doesn't

- ▶ Multi-head attention uses the **same attention mechanism**: score, normalize, aggregate via weighted sum.
- ▶ What changes:
 - ⊙ Each head has its own learned projections of Q, K, V
 - ⊙ These projections map the same inputs into **different representation subspaces**
- ▶ For each word, each head answers the same question:
“What is relevant to me?”, where relevance is learned separately per head.

Multiple Heads

- ▶ Each head uses its own learned linear projections of Q, K, V
- ▶ Multiple **attention heads** run in parallel



Multi-Head Attention (1/2): Dimensions per Head

- ▶ Each head runs **standard scaled dot-product attention**
- ▶ Each head operates on lower-dimensional projections

$$\textcolor{green}{Q}_i \in \mathbb{R}^{|Q| \times \frac{d_k}{h}} \quad \textcolor{brown}{K}_i \in \mathbb{R}^{|K| \times \frac{d_k}{h}} \quad \textcolor{red}{V}_i \in \mathbb{R}^{|V| \times \frac{d_v}{h}}$$

- ▶ Output of each head:

$$\textcolor{black}{Z}_i \in \mathbb{R}^{|Q| \times \frac{d_v}{h}}$$

Multi-Head Attention (2/2): Combining Heads

- ▶ Each head produces a different view of the sequence

$$\mathbf{Z} = \text{Concat}(\mathbf{Z}_1, \mathbf{Z}_2, \dots, \mathbf{Z}_h)$$

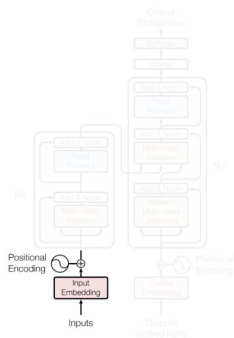
- ▶ Final output shape:

$$\mathbf{Z} \in \mathbb{R}^{|Q| \times d_v}$$

- ▶ Same output dimensionality as single-head attention
 - ⊙ Just more expressive representations, since they combine multiple views

Self-Attention in the Transformer Encoder

Input Processing



Input (Tokenization and) Embedding

Input text is first split into pieces. Can be characters, word, "tokens":

"The detective investigated" -> [The_] [detective_] [invest] [igat] [ed_]

Tokens are indices into the "vocabulary":

[The_] [detective_] [invest] [igat] [ed_] -> [3 721 68 1337 42]

Each vocab entry corresponds to a learned d_{model} -dimensional vector.

[3 721 68 1337 42] -> [[0.123, -5.234, ...], [...], [...], [...], [...]]

Positional Encoding

Remember attention is permutation invariant, but language is not!

("The mouse ate the cat" vs "The cat ate the mouse")

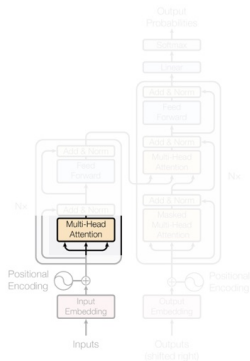
Need to encode position of each word; just add something.

Think [The_] + 10 [detective_] + 20 [invest] + 30 ... but smarter.

Attention: Communication Stage

Multi-headed Self-Attention

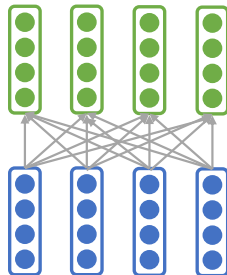
- ▶ Input sequence is used to create queries, keys, and values.
- ▶ Each token “looks around” the whole input, and decides how to update its representation based on what it sees.



Preliminary
contextual word
embedding

Self attention

Word embeddings



What Multi-Head Self-Attention Does *Not* Do

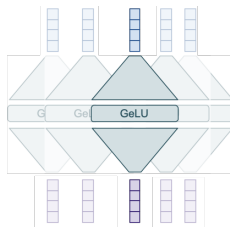
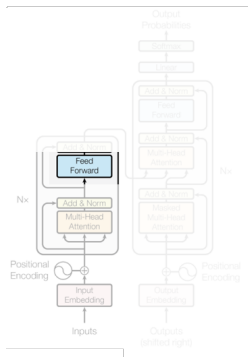
- ▶ Multi-Head Self-Attention is **powerful**:
 - ◉ can model different types of interactions between input tokens
 - ◉ can capture long-range dependencies
- ▶ But Self-Attention is **linear**:
 - ◉ outputs are weighted sums of values
 - ◉ no non-linear transformations across features

To increase representational power, we need non-linear transformations.

Feedforward Layers: Position-wise Computation

- ▶ A **non-linear MLP** applied *independently* to each token
- ▶ Creates new features within each token representation:

$$z_i = W_2 \text{ GeLU}(W_1 x_i + b_1) + b_2$$



- ▶ Bulk of the parameters in giant models, this is what becomes giant.

Feedforward Layers Design Choices

Expansion (feature discovery):

$$\mathbf{h}_i = \mathbf{W}_1 \mathbf{x}_i + \mathbf{b}_1, \quad \mathbf{W}_1 \in \mathbb{R}^{4d \times d}$$

$$\mathbf{h}_i \in \mathbb{R}^{4d}$$

Widening gives the model space to form many intermediate features.

Non-linearity:

$$\mathbf{h}_i \leftarrow \text{GeLU}(\mathbf{h}_i)$$

Projection back (feature selection):

$$\mathbf{z}_i = \mathbf{W}_2 \mathbf{h}_i + \mathbf{b}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{d \times 4d}$$

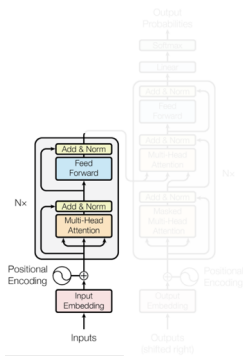
$$\mathbf{z}_i \in \mathbb{R}^d$$

Projecting back forces the model to compress useful features.

From One Block to Depth

- ▶ So far, we have described **one encoder block**
 - ◉ attention: tokens exchange information
 - ◉ feedforward: tokens transform information internally
- ▶ A single block already produces a contextualized representation
 - ◉ but it is still shallow
- ▶ Deep models gain power by **repeating computations** multiple times
 - ◉ each layer refines and recombines features
 - ◉ abstraction increases with depth

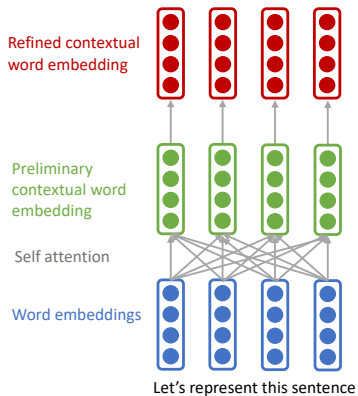
The Encoder: Stacking Blocks



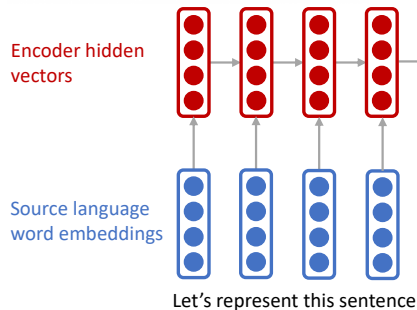
- ▶ Each encoder block preserves shape: input and output are sequences of the same length and dimension
- ▶ The encoder is built by stacking N identical blocks
- ▶ With each layer, representations become more **contextualized and abstract**
- ▶ The final encoder output is a rich sequence representation, used downstream (e.g., by a decoder or task head)

Encoder in a Transformer vs RNN

Transformer Encoder



RNN Encoder



Summary & What's Next

Today: The Transformer Encoder

- ① **Self-attention**: tokens exchange information
- ② **Multi-head attention**: multiple relevance patterns in parallel, without increasing model size
- ③ **Scaled dot-product attention**: stabilizing learning by preventing softmax saturation
- ④ **Feedforward layers**: position-wise, non-linear feature construction
- ⑤ **Encoder blocks**: stacking identical blocks to build deep, contextual representations

Next: Completing the Transformer ...

That's it for today.